Running the ARL

Brendan Griffiths

Contents

1 Overview 2
2 What you are given 2
3 Running with Docker 3
3.1 The compose file 3
3.2 Starting a service 3
3.3 Common settings 4
4 Running the binaries 4
5 Scenarios 5
5.1 File format 5
5.2 Weather events (`service = "env"`) 6
5.3 Service events 6

1 Overview

You can run your own copy of the ARL services for local development and testing. Each service is provided both as a Docker image and as a Linux binary. The Docker images are the recommended way to run them: one command starts a service together with everything it needs.

Every service needs an MQTT broker (Eclipse Mosquitto) and the ARL API, and the API needs a PostgreSQL database. The provided compose file wires these up for you.

Component	**Image**	**Reachable at**
MQTT broker	`eclipse-mosquitto:2`	`localhost:1883` (no username or password)
ARL API	`arl/api:0.1.0`	`http://localhost:8080`, documentation at `http://localhost:8080/docs`
Database	`postgres:16`	Internal only (used by the API)
Radio Telescope Array	`arl/rta:0.1.0`	Topics under `arl/rta/`
Mars Habitation Experiment	`arl/mhe:0.1.0`	Topics under `arl/mhe/`
Local Weather Monitoring	`arl/lwm:0.1.0`	Topics under `arl/lwm/`
On-demand Power Generation	`arl/opg:0.1.0`	Topics under `arl/opg/`
On-demand Water Treatment	`arl/owt:0.1.0`	Topics under `arl/owt/`

The configuration options and scenario events of each service are listed in its own specification (`arl-<service>-spec`, sections *Configuration* and *Scenario events*).

2 What you are given

The ARL bundle unpacks to:

```
arl-sim/
  arl-test.yaml          the compose file (preconfigured, see below)
  config/                one configuration file per component: api.toml, rta.toml,
mhe.toml, ...
  scenarios/             example scenario files
  mosquitto/             broker configuration
  images/                Docker images: arl-api.tar, arl-rta.tar, ...
  bin/                   Linux x86_64 binaries: arl-api, arl-rta, ...
```

Load the Docker images once:

```
docker load -i images/arl-api.tar
docker load -i images/arl-rta.tar     # and arl-mhe, arl-lwm, arl-opg, arl-owt as needed
```

The images contain everything the services need (the site map, the RTA sky images and the API's initial data). The broker and database images (`eclipse-mosquitto:2`, `postgres:16`) are downloaded by Docker the first time you start the ARL.

3 Running with Docker

3.1 The compose file

`arl-test.yaml` is preconfigured to run every component. Run all commands from the `arl-sim/` folder. An excerpt:

```yaml
services:
  api:
    image: arl/api:0.1.0
    depends_on: [postgres, mosquitto]
    volumes: ["./config:/config:ro"]
    ports: ["8080:8080"]

  lwm:
    image: arl/lwm:0.1.0
    depends_on: [mosquitto, api]
    volumes: ["./config:/config:ro", "./scenarios:/scenarios:ro"]
    command: ["--scenario", "${ARL_SCENARIO:-}"]
    environment:
      ARL_SEED: ${ARL_SEED:-1}
      ARL_SPEEDUP: ${ARL_SPEEDUP:-1}
      ARL_START_TIME: ${ARL_START_TIME:-now}
```

Each service reads its configuration from `config/<service>.toml`. Edit that file to change a service's options (see its specification), then restart the service.

3.2 Starting a service

One command per component. Starting a service also starts the broker, the API and the database if they are not running yet.

Component	Command
ARL API (with broker and database)	`docker compose -f arl-test.yaml up -d api`
Radio Telescope Array	`docker compose -f arl-test.yaml up -d rta`
Mars Habitation Experiment	`docker compose -f arl-test.yaml up -d mhe`
Local Weather Monitoring	`docker compose -f arl-test.yaml up -d lwm`
On-demand Power Generation	`docker compose -f arl-test.yaml up -d opg`
On-demand Water Treatment	`docker compose -f arl-test.yaml up -d owt`
Everything	`docker compose -f arl-test.yaml up -d`

Other useful commands:

Task	Command
See what is running and healthy	`docker compose -f arl-test.yaml ps`
Follow a service's log	`docker compose -f arl-test.yaml logs -f lwm`
Restart a service (e.g. after editing its config)	`docker compose -f arl-test.yaml restart lwm`

Task	Command
Stop one service	`docker compose -f arl-test.yaml stop lwm`
Stop everything	`docker compose -f arl-test.yaml down`
Stop everything and reset the API database	`docker compose -f arl-test.yaml down -v`

3.3 Common settings

These settings apply to every service and are given as environment variables in front of the command:

Variable	Default	Meaning
`ARL_SEED`	`1`	Random seed. The same seed, start time, scenario and commands give exactly the same data.
`ARL_SPEEDUP`	`1`	Simulation speed relative to real time (e.g. `10` runs 10 simulated minutes per real minute). Each service has a maximum (`max_speedup` in its configuration): RTA 50, MHE 2000, LWM, OPG and OWT 500.
`ARL_START_TIME`	`now`	Simulated start time: `now` or an RFC 3339 time, e.g. `2027-03-02T08:00:00+02:00`.
`ARL_SCENARIO`	none	Scenario file to load, as a path inside the container, e.g. `/scenarios/example.toml` (see *Scenarios*).

For example, to run the LWM ten times faster with a different seed:

```
ARL_SPEEDUP=10 ARL_SEED=7 docker compose -f arl-test.yaml up -d lwm
```

On Windows PowerShell, set the variables first: `$env:ARL_SPEEDUP = "10"`, then run the command.

On Windows Git Bash, also put `MSYS_NO_PATHCONV=1` in front of the command when a value is a path (such as `ARL_SCENARIO=/scenarios/example.toml`); otherwise Git Bash rewrites the path and the service cannot find the file.

Controls are enabled in your local copy. Commands must carry a researcher code that the ARL API accepts for that service (see the *ARL API* specification).

4 Running the binaries

The binaries are static Linux x86_64 executables (on Windows, run them in WSL). They still need the broker and the API, so start those with Docker first:

```
docker compose -f arl-test.yaml up -d api
```

Then run a service with its configuration file, pointing it at `localhost`:

```
MQTT_BROKER_ADDRESS=localhost ARL_API_ADDRESS=localhost ./bin/arl-lwm --config config/lwm.toml
```

The RTA and LWM binaries need the site map and sky images, and the API binary its initial data. These files are inside the Docker images; copy them out once (after `docker load`):

```
docker create --name arl-files arl/rta:0.1.0 && docker cp arl-files:/assets ./assets &&
docker rm arl-files
docker create --name arl-files arl/api:0.1.0 && docker cp arl-files:/seed ./seed &&
docker rm arl-files
```

The configuration files use container paths (`/assets/...`). When running a binary, copy its configuration file and change `geojson` (RTA, LWM) and `assets_dir` (RTA) to your `assets/` folder; for the API binary set `--seed-dir seed`.

Options every service binary accepts:

Option	Meaning
`--config <file>`	Configuration file (also `ARL_CONFIG`).
`--seed <n>`	Overrides `seed`.
`--speedup <x>`	Overrides `speedup_multiplier`.
`--start-time <now|RFC 3339>`	Overrides `start_time`.
`--control-enabled <true|false>`	Overrides `control_enabled`.
`--scenario <file>`	Overrides `scenario`.
`run`	Run against the broker (the default).
`run --ndjson <file>`	Write every message to a newline-delimited JSON file instead of the broker (no commands are received). Useful for inspecting the data offline.
`healthcheck`	Exit 0 if the running service on this machine is healthy.

Environment variables override the configuration file, and command-line options override both. The full list of variables is in each service's *Configuration* section.

5 Scenarios

A scenario is a file of timed events that makes specific things happen: a thunderstorm, a heatwave, a dish overheating, a pump failing. Marking runs and the live deployment use scenarios to create weather and incidents, and you can use them to test that your platform handles every alert.

Every service reads the same scenario file and applies the events addressed to it. Load a scenario by setting `ARL_SCENARIO` when you start the services:

```
ARL_SCENARIO=/scenarios/example.toml docker compose -f arl-test.yaml up -d lwm rta
```

Put your own scenario files in the `scenarios/` folder; it is mounted at `/scenarios` inside the containers. Restart the services to load a changed file. Events are timed from the start time, so restarting replays the scenario from the beginning.

5.1 File format

A scenario is a TOML file with one `[[event]]` table per event:

```
[[event]]
at = "+00:30:00"          # when: an offset from the start time (+HH:MM:SS), or an RFC
3339 time
```

```toml
service = "env"           # env (the weather), rta, mhe, lwm, opg or owt
fault = "storm"           # what happens (see below and each service specification)
duration = "01:30:00"     # how long (HH:MM:SS); leave out for faults that last until
repaired
params = { gust_ms = 26.0 }

[[event]]
at = "+00:10:00"
service = "rta"
entity = "dish-042"       # the entity it applies to, where the event needs one
fault = "overheat"
duration = "00:20:00"
params = { peak_k = 345.0 }
```

Key	Meaning
`at`	Required. `+HH:MM:SS` after the start time (hours may exceed 24), or an absolute RFC 3339 time.
`service`	Required. `env` for weather, or the service the event is for.
`entity`	The entity id (dish, site, room, crew member, plant entity), where the event needs one.
`fault`	Required. The event type.
`duration`	`HH:MM:SS`. Without it, a fault lasts until a `repair` event or the matching control fixes it.
`params`	Optional parameters of the event.

While a scenario event is active on an entity, the service's own random faults on that entity are paused, so the scenario controls what happens to it.

5.2 Weather events (`service = "env"`)

Weather events change the shared weather model, so every service sees them at the same time (for example, a storm raises LWM wind and lightning alerts, stows the RTA and darkens the OPG solar array).

Fault	Params (default)	Effect
`storm`	`gust_ms` (24 m/s), `speed_ms` (10 m/s, 2 to 30), `rain_mm_h` (30 mm/h, 0 to 100)	A thunderstorm whose gust front reaches the ARL at `at`, coming from the current wind direction at `speed_ms`, with lightning, heavy rain and gusts up to `gust_ms`.
`dust`	none	A dust event for `duration`: stronger wind, less sunshine, faster soiling of solar surfaces.
`heatwave`	`delta_k` (10 K)	Raises the air temperature by `delta_k` for `duration`.
`cold_snap`	`delta_k` (−10 K)	Lowers the air temperature by `delta_k` for `duration`.
`clear_sky`	none	No cloud and no storms for `duration`.

5.3 Service events

Each service specification lists its own events (section *Scenario events*). All services also accept `repair` on an entity, which ends the faults on it. An example using every service is provided in `scenarios/example.toml`.

AI Declaration: Claude Code (Opus 5.5) - generated from the arl-sim source, reviewed